

SQL Complete Notes

CRUD Operations • Joins • Aggregate Functions

Prepared By: Ananda Rimal

August 1, 2026

TOPIC ORDER - WHAT COMES FIRST

1. STEP 1: Create Database & Tables

- CREATE DATABASE
- CREATE TABLE (Table 1: Students, Table 2: Courses)
- INSERT data into tables
- View tables with SELECT * FROM

2. STEP 2: CRUD Operations

- C - CREATE (INSERT)
- R - READ (SELECT with WHERE, ORDER BY, DISTINCT, LIKE, BETWEEN, IN)
- U - UPDATE
- D - DELETE

3. STEP 3: JOINS (Merging Two Tables)

- Create Tables First
- INNER JOIN (Only matching records)
- LEFT JOIN (All from left table)
- RIGHT JOIN (All from right table)
- FULL JOIN (All from both tables)

4. STEP 4: Aggregate Functions

- Create Tables First
- COUNT() - Count rows
- SUM() - Total sum
- AVG() - Average
- MIN() - Minimum
- MAX() - Maximum
- GROUP BY - Group rows
- HAVING - Filter groups

5. STEP 5: Combined Queries

- JOIN + Aggregate Functions together
- Real-world examples

Learning Path: Tables → CRUD → Joins → Aggregates → Combined

Contents

1	Sample Database Tables	4
1.1	Table 1: Students (ALL ROWS)	4
1.2	Table 2: Courses (ALL ROWS)	5
2	CRUD OPERATIONS with Query-to-Table Examples	6
2.1	CREATE - INSERT Operations	6
2.2	READ - SELECT Operations (Each Query with Result Table)	7
2.3	UPDATE Operations (Before/After Tables)	10
2.4	DELETE Operations (Before/After Tables)	11
3	JOINS - Create Tables First, Then Join	12
3.1	Step 1: Create Two Tables for JOIN Practice	12
3.2	Step 2: INNER JOIN - Matching Records Only	14
3.3	Step 3: LEFT JOIN - All Students + Matching Courses	14
3.4	Step 4: RIGHT JOIN - All Courses + Matching Students	15
3.5	Step 5: FULL JOIN - All Records from Both Tables	15
3.6	Join Comparison Summary	16
4	AGGREGATE FUNCTIONS - Create Tables First, Then Test	17
4.1	Step 1: Create Tables for Aggregate Practice	17
4.2	Step 2: COUNT Function	19
4.3	Step 3: SUM Function	19
4.4	Step 4: AVG Function	20
4.5	Step 5: MIN and MAX Functions	20
4.6	Step 6: GROUP BY with Multiple Functions	21
4.7	Step 7: GROUP BY with HAVING	21
5	COMBINED: Joins + Aggregates	23
6	TOPIC ORDER SUMMARY	24
7	QUICK REFERENCE	24

1 Sample Database Tables

1.1 Table 1: Students (ALL ROWS)

Listing 1: Create View Students Table

```
-- Create Students Table
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(50),
    grade CHAR(2)
);

-- Insert All Data
INSERT INTO students VALUES
(1, 'Ram Sharma', 20, 'Kathmandu', 'A'),
(2, 'Sita Rai', 22, 'Pokhara', 'B'),
(3, 'Hari Thapa', 19, 'Kathmandu', 'A'),
(4, 'Gita Magar', 21, 'Chitwan', 'C'),
(5, 'Krishna Gurung', 23, 'Pokhara', 'B');

-- View All Students
SELECT * FROM students;
```

Table 1: Query: **SELECT * FROM students;** Output Table

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	C
5	Krishna Gurung	23	Pokhara	B

1.2 Table 2: Courses (ALL ROWS)

Listing 2: Create View Courses Table

```
-- Create Courses Table
CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(50),
  student_id INT,
  fee DECIMAL(10,2)
);

-- Insert All Data
INSERT INTO courses VALUES
(101, 'Math', 1, 5000),
(102, 'Science', 1, 6000),
(103, 'English', 2, 4500),
(104, 'History', 2, 4000),
(105, 'Math', 3, 5000),
(106, 'Computer', 5, 7000);

-- View All Courses
SELECT * FROM courses;
```

Table 2: Query: **SELECT * FROM courses**; Output Table

Course ID	Course Name	Student ID	Fee
101	Math	1	5000
102	Science	1	6000
103	English	2	4500
104	History	2	4000
105	Math	3	5000
106	Computer	5	7000

2 CRUD OPERATIONS with Query-to-Table Examples

2.1 CREATE - INSERT Operations

Listing 3: INSERT Query Examples

```
-- Insert single row (all columns)
INSERT INTO students VALUES (6, 'Bikash Adhikari', 24, 'Lalitpur',
, 'A');

-- Insert specific columns
INSERT INTO students (student_id, name, age, city)
VALUES (7, 'Anita KC', 20, 'Bhaktapur');

-- Insert multiple rows
INSERT INTO students VALUES
(8, 'Deepak Oli', 22, 'Kathmandu', 'B'),
(9, 'Mina Pandey', 19, 'Pokhara', 'C');

-- Check result after INSERT
SELECT * FROM students;
```

Table 3: Query: **SELECT * FROM students;** (After INSERT) 9 Rows

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	C
5	Krishna Gurung	23	Pokhara	B
6	Bikash Adhikari	24	Lalitpur	A
7	Anita KC	20	Bhaktapur	NULL
8	Deepak Oli	22	Kathmandu	B
9	Mina Pandey	19	Pokhara	C

2.2 READ - SELECT Operations (Each Query with Result Table)

Listing 4: Query: Select specific columns

```
SELECT name, city FROM students;
```

Table 4: Output: SELECT name, city FROM students;

Name	City
Ram Sharma	Kathmandu
Sita Rai	Pokhara
Hari Thapa	Kathmandu
Gita Magar	Chitwan
Krishna Gurung	Pokhara
Bikash Adhikari	Lalitpur
Anita KC	Bhaktapur
Deepak Oli	Kathmandu
Mina Pandey	Pokhara

Listing 5: Query: WHERE clause filter

```
SELECT * FROM students WHERE city = 'Kathmandu';
```

Table 5: Output: WHERE city = 'Kathmandu'

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
3	Hari Thapa	19	Kathmandu	A
8	Deepak Oli	22	Kathmandu	B

Listing 6: Query: WHERE with AND condition

```
SELECT name, age, grade FROM students
WHERE age > 20 AND grade = 'A';
```

Table 6: Output: WHERE age > 20 AND grade = 'A'

Name	Age	Grade
Bikash Adhikari	24	A

Listing 7: Query: ORDER BY (Sorting)

```
SELECT name, age FROM students ORDER BY age DESC;
```

Table 7: Output: ORDER BY age DESC

Name	Age
Bikash Adhikari	24
Krishna Gurung	23
Sita Rai	22
Deepak Oli	22
Gita Magar	21
Ram Sharma	20
Anita KC	20
Hari Thapa	19
Mina Pandey	19

Listing 8: Query: DISTINCT

```
SELECT DISTINCT city FROM students;
```

Table 8: Output: SELECT DISTINCT city

City
Kathmandu
Pokhara
Chitwan
Lalitpur
Bhaktapur

Listing 9: Query: LIKE pattern

```
SELECT * FROM students WHERE name LIKE 'R%';
```

Table 9: Output: WHERE name LIKE 'R%' (Starts with R)

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A

Listing 10: Query: IN operator

```
SELECT * FROM students WHERE city IN ('Kathmandu', 'Pokhara');
```

Table 10: Output: WHERE city IN ('Kathmandu', 'Pokhara')

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
5	Krishna Gurung	23	Pokhara	B
8	Deepak Oli	22	Kathmandu	B
9	Mina Pandey	19	Pokhara	C

Listing 11: Query: BETWEEN

```
SELECT name, age FROM students WHERE age BETWEEN 20 AND 22;
```

Table 11: Output: WHERE age BETWEEN 20 AND 22

Name	Age
Ram Sharma	20
Sita Rai	22
Gita Magar	21
Anita KC	20
Deepak Oli	22

2.3 UPDATE Operations (Before/After Tables)

Listing 12: UPDATE Query: Single Row

```
-- Update grade for student ID 4
UPDATE students SET grade = 'A' WHERE student_id = 4;
```

Table 12: BEFORE UPDATE: Student ID 4

ID	Name	Age	City	Grade
4	Gita Magar	21	Chitwan	C

Table 13: AFTER UPDATE: Student ID 4

ID	Name	Age	City	Grade
4	Gita Magar	21	Chitwan	A

Listing 13: UPDATE Query: Multiple Rows

```
-- Set grade B for all Pokhara students
UPDATE students SET grade = 'B' WHERE city = 'Pokhara';
```

Table 14: AFTER UPDATE: All Pokhara students now grade B

ID	Name	Age	City	Grade
2	Sita Rai	22	Pokhara	B
5	Krishna Gurung	23	Pokhara	B
9	Mina Pandey	19	Pokhara	B

2.4 DELETE Operations (Before/After Tables)

Listing 14: DELETE Query

```
-- Delete student ID 9
DELETE FROM students WHERE student_id = 9;
```

Table 15: **BEFORE DELETE: 9 Students Total**

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	A
5	Krishna Gurung	23	Pokhara	B
6	Bikash Adhikari	24	Lalitpur	A
7	Anita KC	20	Bhaktapur	NULL
8	Deepak Oli	22	Kathmandu	B
9	Mina Pandey	19	Pokhara	B

Table 16: **AFTER DELETE: 8 Students (ID 9 Removed)**

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	A
5	Krishna Gurung	23	Pokhara	B
6	Bikash Adhikari	24	Lalitpur	A
7	Anita KC	20	Bhaktapur	NULL
8	Deepak Oli	22	Kathmandu	B

3 JOINS - Create Tables First, Then Join

3.1 Step 1: Create Two Tables for JOIN Practice

Listing 15: Create Students Table for JOIN

```
-- Drop old table if exists
DROP TABLE IF EXISTS students;

-- Create Students Table
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(50),
    grade CHAR(2)
);

-- Insert Data
INSERT INTO students VALUES
(1, 'Ram Sharma', 20, 'Kathmandu', 'A'),
(2, 'Sita Rai', 22, 'Pokhara', 'B'),
(3, 'Hari Thapa', 19, 'Kathmandu', 'A'),
(4, 'Gita Magar', 21, 'Chitwan', 'C'),
(5, 'Krishna Gurung', 23, 'Pokhara', 'B');

-- View Students Table
SELECT * FROM students;
```

Table 17: Table A: Students (5 Rows)

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	C
5	Krishna Gurung	23	Pokhara	B

Listing 16: Create Courses Table for JOIN

```
-- Drop old table if exists
DROP TABLE IF EXISTS courses;

-- Create Courses Table
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50),
    student_id INT,
    fee DECIMAL(10,2)
);

-- Insert Data
INSERT INTO courses VALUES
(101, 'Math', 1, 5000),
(102, 'Science', 1, 6000),
(103, 'English', 2, 4500),
(104, 'History', 2, 4000),
(105, 'Math', 3, 5000),
(106, 'Computer', 5, 7000);

-- View Courses Table
SELECT * FROM courses;
```

Table 18: Table B: Courses (6 Rows)

Course ID	Course	Student ID	Fee
101	Math	1	5000
102	Science	1	6000
103	English	2	4500
104	History	2	4000
105	Math	3	5000
106	Computer	5	7000

Note: Student ID 4 has NO courses. Course table has no student ID 4.

3.2 Step 2: INNER JOIN - Matching Records Only

INNER JOIN: Returns only matching records from both tables. Student ID 4 (Gita Magar) is EXCLUDED because she has no course.

Listing 17: INNER JOIN Query

```
SELECT *
FROM students AS s
INNER JOIN courses AS c
ON s.student_id = c.student_id;
```

Table 19: INNER JOIN Result (6 Rows - Only Matches)

s.id	s.name	s.age	s.city	s.grade	c.id	c.course	c.fee
1	Ram Sharma	20	Kathmandu	A	101	Math	5000
1	Ram Sharma	20	Kathmandu	A	102	Science	6000
2	Sita Rai	22	Pokhara	B	103	English	4500
2	Sita Rai	22	Pokhara	B	104	History	4000
3	Hari Thapa	19	Kathmandu	A	105	Math	5000
5	Krishna Gurung	23	Pokhara	B	106	Computer	7000

Student ID 4 (Gita Magar) NOT in result - no matching course.

3.3 Step 3: LEFT JOIN - All Students + Matching Courses

LEFT JOIN: Returns ALL students from left table + matching courses. Student ID 4 (Gita Magar) is INCLUDED with NULL values.

Listing 18: LEFT JOIN Query

```
SELECT *
FROM students AS s
LEFT JOIN courses AS c
ON s.student_id = c.student_id;
```

Table 20: LEFT JOIN Result (7 Rows - All Students)

s.id	s.name	s.age	s.city	s.grade	c.id	c.course	c.fee
1	Ram Sharma	20	Kathmandu	A	101	Math	5000
1	Ram Sharma	20	Kathmandu	A	102	Science	6000
2	Sita Rai	22	Pokhara	B	103	English	4500
2	Sita Rai	22	Pokhara	B	104	History	4000
3	Hari Thapa	19	Kathmandu	A	105	Math	5000
4	Gita Magar	21	Chitwan	C	NULL	NULL	NULL
5	Krishna Gurung	23	Pokhara	B	106	Computer	7000

Student ID 4 (Gita Magar) INCLUDED with NULL - no course but still shown.

3.4 Step 4: RIGHT JOIN - All Courses + Matching Students

RIGHT JOIN: Returns ALL courses from right table + matching students. All 6 courses are INCLUDED.

Listing 19: RIGHT JOIN Query

```
SELECT *
FROM students AS s
RIGHT JOIN courses AS c
ON s.student_id = c.student_id;
```

Table 21: RIGHT JOIN Result (6 Rows - All Courses)

s.id	s.name	s.age	s.city	s.grade	c.id	c.course	c.fee
1	Ram Sharma	20	Kathmandu	A	101	Math	5000
1	Ram Sharma	20	Kathmandu	A	102	Science	6000
2	Sita Rai	22	Pokhara	B	103	English	4500
2	Sita Rai	22	Pokhara	B	104	History	4000
3	Hari Thapa	19	Kathmandu	A	105	Math	5000
5	Krishna Gurung	23	Pokhara	B	106	Computer	7000

All 6 courses are shown with their student details.

3.5 Step 5: FULL JOIN - All Records from Both Tables

FULL JOIN: Returns ALL records from both tables. NULL values for non-matching rows from either side. (MySQL uses UNION of LEFT and RIGHT JOIN)

Listing 20: FULL JOIN Query (MySQL)

```
SELECT *
FROM students AS s
LEFT JOIN courses AS c
ON s.student_id = c.student_id

UNION

SELECT *
FROM students AS s
RIGHT JOIN courses AS c
ON s.student_id = c.student_id;
```

Student 4 included (no course) + All courses included.

Table 22: FULL JOIN Result (7 Rows - All Records from Both)

s.id	s.name	s.age	s.city	s.grade	c.id	c.course	c.fee
1	Ram Sharma	20	Kathmandu	A	101	Math	5000
1	Ram Sharma	20	Kathmandu	A	102	Science	6000
2	Sita Rai	22	Pokhara	B	103	English	4500
2	Sita Rai	22	Pokhara	B	104	History	4000
3	Hari Thapa	19	Kathmandu	A	105	Math	5000
4	Gita Magar	21	Chitwan	C	NULL	NULL	NULL
5	Krishna Gurung	23	Pokhara	B	106	Computer	7000

3.6 Join Comparison Summary

Table 23: Four JOIN Types - Side by Side Comparison

JOIN Type	Total Rows	Student 4	What It Returns
INNER JOIN	6 rows	Excluded	Only matching records
LEFT JOIN	7 rows	NULL values	All left + matching right
RIGHT JOIN	6 rows	Excluded	All right + matching left
FULL JOIN	7 rows	NULL values	All from both tables

Table 24: JOIN Syntax Summary - Simple Code Structure

Type	SQL Syntax
INNER JOIN	SELECT * FROM A AS a INNER JOIN B AS b ON a.id = b.id
LEFT JOIN	SELECT * FROM A AS a LEFT JOIN B AS b ON a.id = b.id
RIGHT JOIN	SELECT * FROM A AS a RIGHT JOIN B AS b ON a.id = b.id
FULL JOIN	LEFT JOIN + UNION + RIGHT JOIN

4 AGGREGATE FUNCTIONS - Create Tables First, Then Test

4.1 Step 1: Create Tables for Aggregate Practice

Listing 21: Create Students Table for Aggregates

```
-- Drop old table if exists
DROP TABLE IF EXISTS students;

-- Create Students Table
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(50),
    grade CHAR(2)
);

-- Insert Data
INSERT INTO students VALUES
(1, 'Ram Sharma', 20, 'Kathmandu', 'A'),
(2, 'Sita Rai', 22, 'Pokhara', 'B'),
(3, 'Hari Thapa', 19, 'Kathmandu', 'A'),
(4, 'Gita Magar', 21, 'Chitwan', 'C'),
(5, 'Krishna Gurung', 23, 'Pokhara', 'B');

-- View Students Table
SELECT * FROM students;
```

Table 25: Students Table (5 Rows)

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	C
5	Krishna Gurung	23	Pokhara	B

Listing 22: Create Courses Table for Aggregates

```
-- Drop old table if exists
DROP TABLE IF EXISTS courses;

-- Create Courses Table
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50),
    student_id INT,
    fee DECIMAL(10,2)
);

-- Insert Data
INSERT INTO courses VALUES
(101, 'Math', 1, 5000),
(102, 'Science', 1, 6000),
(103, 'English', 2, 4500),
(104, 'History', 2, 4000),
(105, 'Math', 3, 5000),
(106, 'Computer', 5, 7000);

-- View Courses Table
SELECT * FROM courses;
```

Table 26: Courses Table (6 Rows)

Course ID	Course Name	Student ID	Fee
101	Math	1	5000
102	Science	1	6000
103	English	2	4500
104	History	2	4000
105	Math	3	5000
106	Computer	5	7000

4.2 Step 2: COUNT Function

Listing 23: COUNT Query Examples

```
-- Count all students
SELECT COUNT(*) AS total_students FROM students;

-- Count by grade
SELECT grade, COUNT(*) AS student_count
FROM students
GROUP BY grade;
```

Table 27: Output: COUNT(*)

total_students
5

Table 28: Output: COUNT by Grade

Grade	student_count
A	2
B	2
C	1

4.3 Step 3: SUM Function

Listing 24: SUM Query Examples

```
-- Total fees collected
SELECT SUM(fee) AS total_fees FROM courses;

-- Sum by course name
SELECT course_name, SUM(fee) AS total_collection
FROM courses GROUP BY course_name;
```

Table 29: Output: SUM(fee)

total_fees
31500

Table 30: Output: SUM by Course

Course	total_collection
Computer	7000
English	4500
History	4000
Math	10000
Science	6000

4.4 Step 4: AVG Function

Listing 25: AVG Query Examples

```
-- Average student age
SELECT AVG(age) AS avg_age FROM students;

-- Average by city
SELECT city, AVG(age) AS avg_age
FROM students GROUP BY city;
```

Table 31: Output: AVG(age)

avg_age
21.00

Table 32: Output: AVG by City

City	avg_age
Chitwan	21.00
Kathmandu	19.50
Pokhara	22.50

4.5 Step 5: MIN and MAX Functions

Listing 26: MIN and MAX Query Examples

```
-- Youngest and oldest student
SELECT MIN(age) AS youngest, MAX(age) AS oldest FROM students;

-- Min and Max fees by course
SELECT course_name, MIN(fee) AS min_fee, MAX(fee) AS max_fee
FROM courses GROUP BY course_name;
```

Table 33: Output: MIN & MAX Age

youngest	oldest
19	23

Table 34: Output: Min/Max Fee by Course

Course	min_fee	max_fee
Computer	7000	7000
English	4500	4500
History	4000	4000
Math	5000	5000
Science	6000	6000

4.6 Step 6: GROUP BY with Multiple Functions

Listing 27: All Aggregates Together

```
-- Complete statistics by city
SELECT
    city,
    COUNT(*) AS total_students,
    SUM(age) AS sum_age,
    AVG(age) AS avg_age,
    MIN(age) AS youngest,
    MAX(age) AS oldest
FROM students
GROUP BY city;
```

Table 35: Output: Complete Aggregate by City

City	total	sum	avg	min	max
Kathmandu	2	39	19.50	19	20
Pokhara	2	45	22.50	22	23
Chitwan	1	21	21.00	21	21

4.7 Step 7: GROUP BY with HAVING

Listing 28: HAVING Query Example

```
-- Cities with more than 1 student
SELECT city, COUNT(*) AS total
FROM students
GROUP BY city
HAVING COUNT(*) > 1;

-- WHERE vs HAVING
-- WHERE filters rows BEFORE grouping
-- HAVING filters groups AFTER grouping
```

```
SELECT city, COUNT(*) AS total
FROM students
WHERE age >= 20
GROUP BY city
HAVING COUNT(*) > 1;
```

Table 36: **Output: HAVING COUNT(*) ≥ 1**

City	total
Kathmandu	2
Pokhara	2

5 COMBINED: Joins + Aggregates

Listing 29: Query: Students with total fees and course count

```
SELECT
    s.student_id,
    s.name,
    COUNT(c.course_id) AS courses_enrolled,
    SUM(c.fee) AS total_fees_paid
FROM students AS s
LEFT JOIN courses AS c ON s.student_id = c.student_id
GROUP BY s.student_id, s.name
ORDER BY total_fees_paid DESC;
```

Table 37: Output: JOIN + Aggregate Result

ID	Name	Courses	Total Fees
1	Ram Sharma	2	11000
2	Sita Rai	2	8500
5	Krishna Gurung	1	7000
3	Hari Thapa	1	5000
4	Gita Magar	0	0

Listing 30: Query: City-wise course enrollment and revenue

```
SELECT
    s.city,
    COUNT(c.course_id) AS total_courses,
    SUM(c.fee) AS total_revenue
FROM students AS s
LEFT JOIN courses AS c ON s.student_id = c.student_id
GROUP BY s.city
ORDER BY total_revenue DESC;
```

Table 38: Output: City-wise Course Revenue

City	total_courses	revenue
Kathmandu	3	16000
Pokhara	3	15500
Chitwan	0	NULL

6 TOPIC ORDER SUMMARY

COMPLETE TOPIC ORDER - FROM START TO END

Step	Topic	What You Learn
1	Create Tables	CREATE TABLE, INSERT data
2	CRUD Operations	INSERT, SELECT, UPDATE, DELETE
3	SELECT Filters	WHERE, ORDER BY, DISTINCT, LIKE, BETWEEN, IN
4	JOINS	Create tables, then INNER, LEFT, RIGHT, FULL JOIN
5	Aggregate Functions	Create tables, then COUNT, SUM, AVG, MIN, MAX
6	GROUP BY & HAVING	Grouping rows, filtering groups
7	Combined Queries	JOIN + Aggregates together

Learning Flow:

Create Tables → Insert Data → Read Data → Update/Delete

↓

Create Tables → Join Tables → Aggregate Data → Combine Everything

7 QUICK REFERENCE

CRUD OPERATIONS	
C - CREATE	INSERT INTO table VALUES (v1, v2);
R - READ	SELECT col FROM table WHERE condition;
U - UPDATE	UPDATE table SET col=val WHERE condition;
D - DELETE	DELETE FROM table WHERE condition;

JOINS QUICK REFERENCE		
JOIN	Returns	Example
INNER	Only matches	5+6 tables = 6 rows
LEFT	All left table	5+6 tables = 7 rows
RIGHT	All right table	5+6 tables = 6 rows
FULL	All both tables	5+6 tables = 7 rows

AGGREGATE FUNCTIONS	
COUNT(*)	Count rows
SUM(col)	Sum values
AVG(col)	Average values
MIN(col)	Minimum value
MAX(col)	Maximum value
GROUP BY col	Group rows
HAVING condition	Filter groups

Prepared By

Ananda Rimal

Database Management System Notes

August 1, 2026